

# Smart Agriculture Robot

Full Code, Pin Diagram, and Step-by-Step Wiring Guide

## Project Summary

This guide is for an ESP32 based Smart Agriculture Robot with WiFi dashboard control, manual and auto modes, two relays named Crops Cutting and Spray Water, IR based auto movement, soil moisture sensing, rain sensing, ultrasonic obstacle detection, DHT22 temperature and humidity sensing, and a 16x2 I2C LCD display.

WiFi dashboard name: **Smart Agriculture Robot**. Password: **12345678**. Browser address after connecting: **192.168.4.1**.

## Important Safety Notes

| Area            | Instruction                                                                                                              |
|-----------------|--------------------------------------------------------------------------------------------------------------------------|
| Common GND      | All modules, ESP32 GND, motor driver GND, relay GND, and battery negative must be connected together.                    |
| ESP32 ADC pins  | Soil and rain analog outputs must not exceed 3.3V. Power these sensor modules from 3.3V when possible.                   |
| Ultrasonic ECHO | HC-SR04 ECHO is usually 5V. Use a voltage divider before ESP32 GPIO 23.                                                  |
| LCD I2C         | If LCD is powered by 5V, use an I2C level shifter for SDA/SCL, or confirm the pull-ups are not pulling ESP32 pins to 5V. |
| Motor power     | Do not power motors from the ESP32 3.3V pin. Use a separate motor battery through a motor driver.                        |

## Full ESP32 Pin Diagram

| Module / Part   | Module Pin  | ESP32 Pin / Power | Notes                            |
|-----------------|-------------|-------------------|----------------------------------|
| Motor driver    | IN1         | GPIO 18           | Motor control input              |
| Motor driver    | IN2         | GPIO 19           | Motor control input              |
| Motor driver    | IN3         | GPIO 21           | Motor control input              |
| Motor driver    | IN4         | GPIO 22           | Motor control input              |
| Relay 1         | IN          | GPIO 25           | Dashboard name: Crops Cutting    |
| Relay 2         | IN          | GPIO 26           | Dashboard name: Spray Water      |
| Left IR sensor  | OUT / D0    | GPIO 32           | IR auto movement input           |
| Right IR sensor | OUT / D0    | GPIO 33           | IR auto movement input           |
| Soil sensor     | AO          | GPIO 34           | Analog input only                |
| Rain sensor     | AO          | GPIO 35           | Analog input only                |
| Ultrasonic      | TRIG        | GPIO 5            | Output trigger                   |
| Ultrasonic      | ECHO        | GPIO 23           | Use voltage divider              |
| DHT22           | DATA        | GPIO 27           | 10k pull-up to 3.3V if needed    |
| 16x2 I2C LCD    | SDA         | GPIO 16           | Custom I2C SDA                   |
| 16x2 I2C LCD    | SCL         | GPIO 17           | Custom I2C SCL                   |
| All sensors     | VCC         | 3.3V preferred    | Protect ESP32 input pins         |
| Relay module    | VCC         | 5V                | Common GND with ESP32            |
| Motor driver    | Motor power | External battery  | Do not use ESP32 3.3V for motors |

# Simplified Pin Diagram

| ESP32 DEVKIT  |      |                                |  |      |                                     |
|---------------|------|--------------------------------|--|------|-------------------------------------|
| +-----+-----+ |      |                                |  |      |                                     |
| Motor IN1     | <--- | GPIO 18                        |  | ---> | GPIO 25 Relay 1 IN (Crops Cutting)  |
| Motor IN2     | <--- | GPIO 19                        |  | ---> | GPIO 26 Relay 2 IN (Spray Water)    |
| Motor IN3     | <--- | GPIO 21                        |  | ---> | GPIO 27 DHT22 DATA                  |
| Motor IN4     | <--- | GPIO 22                        |  | ---> | GPIO 16 LCD SDA                     |
| Left IR       | ---> | GPIO 32                        |  | ---> | GPIO 17 LCD SCL                     |
| Right IR      | ---> | GPIO 33                        |  | ---> | GPIO 5 Ultrasonic TRIG              |
| Soil A0       | ---> | GPIO 34                        |  | <--- | GPIO 23 Ultrasonic ECHO via divider |
| Rain A0       | ---> | GPIO 35                        |  |      |                                     |
|               |      | 3.3V -> sensors/DHT where safe |  |      |                                     |
|               |      | 5V/VIN -> relay/LCD if needed  |  |      |                                     |
|               |      | GND -> common ground           |  |      |                                     |
| +-----+-----+ |      |                                |  |      |                                     |

# Step-by-Step Connection Guide

## Step 1 - Power and common ground

Connect ESP32 GND, motor driver GND, relay module GND, all sensor GND pins, and battery negative together. This common ground is required for stable signals.

## Step 2 - Motor driver connection

Connect ESP32 GPIO 18, 19, 21, and 22 to motor driver IN1, IN2, IN3, and IN4. Connect motors to the motor driver output terminals. Connect motor driver motor-power input to a suitable external battery.

## Step 3 - Relay modules

Relay 1 IN goes to GPIO 25 and is named Crops Cutting in the dashboard. Relay 2 IN goes to GPIO 26 and is named Spray Water. Connect relay VCC to 5V and relay GND to common GND. Use COM and NO terminals for normal ON/OFF loads.

## Step 4 - IR sensors

Left IR OUT goes to GPIO 32. Right IR OUT goes to GPIO 33. VCC should be 3.3V if the module output follows VCC level. GND goes to common GND. Auto mode logic: no IR detected = stop, both detected = forward, right detected = turn right, left detected = turn left.

## Step 5 - Soil moisture sensor

Connect soil sensor AO to GPIO 34. Connect VCC to 3.3V and GND to common GND. If the percentage is reversed, swap SOIL\_DRY\_VALUE and SOIL\_WET\_VALUE in the code.

## Step 6 - Rain sensor

Connect rain sensor AO to GPIO 35. Connect VCC to 3.3V and GND to common GND. If rain percentage is reversed, swap RAIN\_DRY\_VALUE and RAIN\_WET\_VALUE in the code.

## Step 7 - Ultrasonic sensor

Connect TRIG to GPIO 5. Connect ECHO to GPIO 23 through a voltage divider. Example: ECHO -> 1k resistor -> GPIO 23, then GPIO 23 -> 2k resistor -> GND. Connect VCC to 5V and GND to common GND.

## Step 8 - DHT22 temperature and humidity sensor

Connect DATA to GPIO 27, VCC to 3.3V, and GND to common GND. If your DHT22 is not a module, add a 10k resistor between DATA and 3.3V.

## Step 9 - 16x2 I2C LCD

Connect LCD SDA to GPIO 16 and SCL to GPIO 17. Connect LCD VCC and GND. If using 5V LCD module, use an I2C level shifter or make sure SDA/SCL are not pulled to 5V.

## Step 10 - Upload and connect

Upload the code to ESP32. Connect phone or laptop WiFi to Smart Agriculture Robot. Password is 12345678. Open browser at 192.168.4.1.

## Motor Direction Fix

This version swaps the forward and backward motor functions because your robot moved backward when Forward was pressed and moved forward when Backward was pressed. If it still moves incorrectly after uploading, the easiest hardware fix is to swap the two wires of the motor that is reversed.

## Testing Checklist

| Test                 | Expected Result                                                                                |
|----------------------|------------------------------------------------------------------------------------------------|
| Connect WiFi         | Network name should appear as Smart Agriculture Robot.                                         |
| Open dashboard       | Browser address 192.168.4.1 should load the dashboard.                                         |
| Forward button       | Robot should move forward.                                                                     |
| Backward button      | Robot should move backward.                                                                    |
| Crops Cutting button | Relay 1 should turn ON/OFF.                                                                    |
| Spray Water button   | Relay 2 should turn ON/OFF.                                                                    |
| Auto mode            | Robot follows IR sensor logic and stops for close obstacles.                                   |
| LCD                  | LCD cycles through temperature, humidity, rain, object distance, soil, mode, and relay status. |
| DHT22                | Dashboard and LCD show temperature and humidity.                                               |

# Full Arduino Code

Copy this complete sketch into Arduino IDE. Install the required libraries first: ArduinoJson, WebSockets by Markus Sattler, LiquidCrystal\_I2C, DHT sensor library by Adafruit, and Adafruit Unified Sensor.

```
#include <WiFi.h>
#include <WebServer.h>
#include <WebSocketsServer.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// ----- WiFi AP -----
const char* ssid = "Smart Agriculture Robot";
const char* password = "12345678";

// ----- Motor Pins -----
#define IN1 18
#define IN2 19
#define IN3 21
#define IN4 22

// ----- Relay Pins -----
#define RELAY1_PIN 25
#define RELAY2_PIN 26

// ----- IR Sensor Pins -----
#define LEFT_IR_PIN 32
#define RIGHT_IR_PIN 33

// ----- Soil Sensor Pin -----
#define SOIL_SENSOR_PIN 34

// ----- Rain Sensor Pin -----
#define RAIN_SENSOR_PIN 35

// ----- Ultrasonic Sensor Pins -----
#define ULTRASONIC_TRIG_PIN 5
#define ULTRASONIC_ECHO_PIN 23

// ----- DHT22 Sensor -----
#define DHT_PIN 27
#define DHT_TYPE DHT22
DHT dht(DHT_PIN, DHT_TYPE);

// ----- 16x2 I2C LCD -----
// GPIO 21 and GPIO 17 are already used by motor, so LCD uses GPIO 16 and GPIO 17.
#define I2C_SDA 16
#define I2C_SCL 17

// Most I2C LCD address is 0x27. If LCD is blank, try 0x3F.
#define LCD_ADDR 0x27
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);

// ----- Settings -----
#define OBSTACLE_STOP_DISTANCE_CM 15

#define SOIL_DRY_VALUE 3500
#define SOIL_WET_VALUE 1300

#define RAIN_DRY_VALUE 4095
#define RAIN_WET_VALUE 1500

// Most IR sensors output LOW when detected.
// If your IR sensor gives HIGH when detected, change LOW to HIGH.
#define IR_DETECTED_STATE LOW

// Most relay modules are Active LOW.
// If relay works opposite, swap LOW and HIGH.
#define RELAY_ON LOW
#define RELAY_OFF HIGH

WebServer server(80);
WebSocketsServer webSocket = WebSocketsServer(81);

// ----- Variables -----
String motorStatus = "Stopped";

bool relay1State = false;
bool relay2State = false;
bool autoMode = false;

bool leftIRDetected = false;
bool rightIRDetected = false;

int soilRawValue = 0;
int soilMoisturePercent = 0;
String soilStatus = "Unknown";

int rainRawValue = 0;
int rainPercent = 0;
String rainStatus = "Unknown";

float distanceCm = -1;
String ultrasonicStatus = "Unknown";

float temperatureC = 0;
float humidityPercent = 0;
String dhtStatus = "Unknown";

unsigned long lastSensorReadTime = 0;
unsigned long lastDashboardSendTime = 0;
unsigned long lastDHTReadTime = 0;
unsigned long lastLCDUpdateTime = 0;
int lcdPage = 0;

// ----- Function Prototypes -----
void sendStatus(uint8_t clientNum);
String HTMLPage();

// ----- LCD Helper -----
String fit16(String text) {
    if (text.length() > 16) {
        return text.substring(0, 16);
    }

    while (text.length() < 16) {
        text += " ";
    }

    return text;
}

// ----- Motor Functions -----
```

```

void stopCar() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);

  motorStatus = "Stopped";
}

// Forward and backward were swapped from the earlier version.
// If your robot still moves opposite, swap the HIGH/LOW values here again.
void moveForward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);

  motorStatus = "Moving Forward";
}

void moveBackward() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);

  motorStatus = "Moving Backward";
}

void turnLeft() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);

  motorStatus = "Turning Left";
}

void turnRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);

  motorStatus = "Turning Right";
}

// ----- Relay Functions -----
void setRelay1(bool state) {
  relay1State = state;
  digitalWrite(RELAY1_PIN, relay1State ? RELAY_ON : RELAY_OFF);
}

void setRelay2(bool state) {
  relay2State = state;
  digitalWrite(RELAY2_PIN, relay2State ? RELAY_ON : RELAY_OFF);
}

// ----- Sensor Functions -----
void readIRSensors() {
  leftIRDetected = digitalRead(LEFT_IR_PIN) == IR_DETECTED_STATE;
  rightIRDetected = digitalRead(RIGHT_IR_PIN) == IR_DETECTED_STATE;
}

void readSoilSensor() {
  soilRawValue = analogRead(SOIL_SENSOR_PIN);

  soilMoisturePercent = map(
    soilRawValue,
    SOIL_DRY_VALUE,
    SOIL_WET_VALUE,
    0,
    100
  );

  soilMoisturePercent = constrain(soilMoisturePercent, 0, 100);

  if (soilMoisturePercent < 30) {
    soilStatus = "Dry";
  }
  else if (soilMoisturePercent < 70) {
    soilStatus = "Normal";
  }
  else {
    soilStatus = "Wet";
  }
}

void readRainSensor() {
  rainRawValue = analogRead(RAIN_SENSOR_PIN);

  rainPercent = map(
    rainRawValue,
    RAIN_DRY_VALUE,
    RAIN_WET_VALUE,
    0,
    100
  );

  rainPercent = constrain(rainPercent, 0, 100);

  if (rainPercent < 20) {
    rainStatus = "No Rain";
  }
  else if (rainPercent < 60) {
    rainStatus = "Light Rain";
  }
  else {
    rainStatus = "Heavy Rain";
  }
}

void readUltrasonicSensor() {
  digitalWrite(ULTRASONIC_TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(ULTRASONIC_TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(ULTRASONIC_TRIG_PIN, LOW);

  // Short timeout helps keep dashboard stable.
  long duration = pulseIn(ULTRASONIC_ECHO_PIN, HIGH, 20000);

  if (duration == 0) {
    distanceCm = -1;
    ultrasonicStatus = "Out of Range";
  }
}

```

```

else {
    distanceCm = duration * 0.0343 / 2.0;

    if (distanceCm <= OBSTACLE_STOP_DISTANCE_CM) {
        ultrasonicStatus = "Obstacle Close";
    }
    else {
        ultrasonicStatus = "Clear";
    }
}
}

void readDHT22Sensor() {
    // DHT22 should not be read too fast.
    if (millis() - lastDHTReadTime < 2500 && dhtStatus != "Unknown") {
        return;
    }

    lastDHTReadTime = millis();

    float h = dht.readHumidity();
    float t = dht.readTemperature();

    if (isnan(h) || isnan(t)) {
        dhtStatus = "DHT Error";
        return;
    }

    humidityPercent = h;
    temperatureC = t;
    dhtStatus = "OK";
}

void readAllSensors() {
    readIRSensors();
    readSoilSensor();
    readRainSensor();
    readUltrasonicSensor();
    readDHT22Sensor();
}

// ----- Auto Mode Logic -----
void autoModeControl() {
    if (!autoMode) {
        return;
    }

    String oldMotorStatus = motorStatus;

    if (distanceCm > 0 && distanceCm <= OBSTACLE_STOP_DISTANCE_CM) {
        stopCar();
        motorStatus = "Obstacle Stopped";

        if (oldMotorStatus != motorStatus) {
            sendStatus(255);
        }
    }

    return;

    if (!leftIRDetected && !rightIRDetected) {
        stopCar();
    }

    else if (leftIRDetected && rightIRDetected) {
        moveForward();
    }

    else if (!leftIRDetected && rightIRDetected) {
        turnRight();
    }

    else if (leftIRDetected && !rightIRDetected) {
        turnLeft();
    }

    if (oldMotorStatus != motorStatus) {
        sendStatus(255);
    }
}

// ----- LCD Display -----
void updateLCDDisplay() {
    if (millis() - lastLCDUpdateTime < 2500) {
        return;
    }

    lastLCDUpdateTime = millis();

    lcd.clear();

    if (lcdPage == 0) {
        lcd.setCursor(0, 0);
        lcd.print(fit16("Temp:" + String(temperatureC, 1) + "C"));

        lcd.setCursor(0, 1);
        lcd.print(fit16("Hum : " + String(humidityPercent, 0) + "%"));
    }

    else if (lcdPage == 1) {
        lcd.setCursor(0, 0);
        lcd.print(fit16("Rain:" + rainStatus));

        lcd.setCursor(0, 1);
        if (distanceCm < 0) {
            lcd.print(fit16("Obj:Out Range"));
        }
        else {
            lcd.print(fit16("Obj:" + String(distanceCm, 1) + "cm"));
        }
    }

    else if (lcdPage == 2) {
        lcd.setCursor(0, 0);
        lcd.print(fit16("Soil:" + String(soilMoisturePercent) + "%"));

        lcd.setCursor(0, 1);
        lcd.print(fit16("Status:" + soilStatus));
    }

    else if (lcdPage == 3) {
        lcd.setCursor(0, 0);
        lcd.print(fit16(autoMode ? "Mode:AUTO" : "Mode:MANUAL"));

        lcd.setCursor(0, 1);
        lcd.print(fit16(motorStatus));
    }

    else if (lcdPage == 4) {

```

```

    lcd.setCursor(0, 0);
    lcd.print(fit16("LIR:" + String(leftIRDetected ? "YES" : "NO")));

    lcd.setCursor(0, 1);
    lcd.print(fit16("RIR:" + String(rightIRDetected ? "YES" : "NO")));
}

else if (lcdPage == 5) {
    lcd.setCursor(0, 0);
    lcd.print(fit16("Cut:" + String(relay1State ? "ON" : "OFF")));

    lcd.setCursor(0, 1);
    lcd.print(fit16("Spray:" + String(relay2State ? "ON" : "OFF")));
}

lcdPage++;

if (lcdPage > 5) {
    lcdPage = 0;
}
}

// ----- Send Status to Dashboard -----
void sendStatus(uint8_t clientNum) {
    StaticJsonDocument<2048> doc;

    doc["motor"] = motorStatus;
    doc["autoMode"] = autoMode;

    doc["relay1"] = relay1State;
    doc["relay2"] = relay2State;

    doc["leftIR"] = leftIRDetected;
    doc["rightIR"] = rightIRDetected;

    doc["soilRaw"] = soilRawValue;
    doc["soilPercent"] = soilMoisturePercent;
    doc["soilStatus"] = soilStatus;

    doc["rainRaw"] = rainRawValue;
    doc["rainPercent"] = rainPercent;
    doc["rainStatus"] = rainStatus;

    doc["distanceCm"] = distanceCm;
    doc["ultrasonicStatus"] = ultrasonicStatus;

    doc["temperatureC"] = temperatureC;
    doc["humidity"] = humidityPercent;
    doc["dhtStatus"] = dhtStatus;

    String jsonString;
    serializeJson(doc, jsonString);

    if (clientNum == 255) {
        websocket.broadcastTXT(jsonString);
    } else {
        websocket.sendTXT(clientNum, jsonString);
    }
}

// ----- WebSocket Event -----
void websocketEvent(uint8_t num, WStype_t type, uint8_t * payload, size_t length) {
    if (type == WStype_DISCONNECTED) {
        Serial.printf("[%u] Disconnected\n", num);

        if (!autoMode) {
            stopCar();
        }
    }

    else if (type == WStype_CONNECTED) {
        IPAddress ip = websocket.remoteIP(num);

        Serial.printf("[%u] Connected from %d.%d.%d.%d\n",
            num, ip[0], ip[1], ip[2], ip[3]);

        sendStatus(num);
    }

    else if (type == WStype_TEXT) {
        StaticJsonDocument<256> doc;
        DeserializationError error = deserializeJson(doc, payload, length);

        if (error) {
            Serial.print("JSON Error: ");
            Serial.println(error.f_str());
            return;
        }

        if (doc.containsKey("autoMode")) {
            autoMode = doc["autoMode"];
            stopCar();

            if (autoMode) {
                motorStatus = "Auto Mode Started";
            } else {
                motorStatus = "Manual Mode Started";
            }
        }

        if (doc.containsKey("motor") && !autoMode) {
            const char* motorCommand = doc["motor"];

            if (strcmp(motorCommand, "forward") == 0) {
                moveForward();
            }

            else if (strcmp(motorCommand, "backward") == 0) {
                moveBackward();
            }

            else if (strcmp(motorCommand, "left") == 0) {
                turnLeft();
            }

            else if (strcmp(motorCommand, "right") == 0) {
                turnRight();
            }

            else if (strcmp(motorCommand, "stop") == 0) {
                stopCar();
            }
        }

        if (doc.containsKey("relay1")) {
            bool state = doc["relay1"];
            setRelay1(state);
        }
    }
}

```

```

    }

    if (doc.containsKey("relay2")) {
        bool state = doc["relay2"];
        setRelay2(state);
    }

    sendStatus(255);
}
}

// ----- Web Dashboard Page -----
String HTMLPage() {
    String page = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Smart Agriculture Robot</title>

<style>
:root {
    --bg: #f4f7f1;
    --card: #ffffff;
    --green: #1f7a4d;
    --green-dark: #145c38;
    --blue: #2563eb;
    --orange: #f97316;
    --red: #dc2626;
    --gray: #64748b;
    --text: #102016;
    --muted: #5f6f64;
    --border: #d8e6dc;
}

* {
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background: var(--bg);
    color: var(--text);
    margin: 0;
    padding: 0;
}

.header {
    background: linear-gradient(135deg, #145c38, #1f7a4d);
    color: white;
    padding: 22px 16px;
    text-align: center;
}

.header h1 {
    margin: 0;
    font-size: 26px;
}

.header p {
    margin: 8px 0 0;
    color: #e7f5ec;
}

.container {
    width: 94%;
    max-width: 1100px;
    margin: 18px auto 30px;
}

#connectionStatus {
    padding: 12px;
    border-radius: 12px;
    margin-bottom: 16px;
    text-align: center;
    font-weight: bold;
}

.connected {
    background: #dcfce7;
    color: #166534;
    border: 1px solid #86efac;
}

.disconnected {
    background: #fee2e2;
    color: #991b1b;
    border: 1px solid #fecaca;
}

.grid {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 16px;
}

.card {
    background: var(--card);
    border: 1px solid var(--border);
    border-radius: 18px;
    padding: 18px;
    box-shadow: 0 4px 14px rgba(0,0,0,0.06);
}

.card h2 {
    margin: 0 0 12px;
    font-size: 20px;
    color: var(--green-dark);
}

.status {
    color: var(--green);
    font-weight: bold;
}

.row {
    display: flex;
    justify-content: space-between;
    gap: 12px;
    border-bottom: 1px solid #eef4ef;
    padding: 8px 0;
}

.row:last-child {
    border-bottom: none;
}

```

```

}

button {
  border: none;
  border-radius: 12px;
  padding: 12px 18px;
  font-size: 17px;
  cursor: pointer;
  margin: 5px;
  color: white;
  font-weight: bold;
  user-select: none;
}

button:active {
  transform: scale(0.96);
}

.modeBtn {
  width: 100%;
  background: var(--blue);
}

.autoMode {
  background: var(--orange);
}

.manualMode {
  background: var(--blue);
}

.control-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 8px;
  align-items: center;
}

.move {
  width: 100%;
  min-height: 58px;
  background: var(--green);
}

.stopBtn {
  background: var(--red);
}

.empty {
  visibility: hidden;
}

.relay-wrap {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 12px;
}

.relayBtn {
  width: 100%;
  min-height: 54px;
}

.relayOn {
  background: var(--green);
}

.relayOff {
  background: var(--gray);
}

.disabledBtn {
  background: var(--gray) !important;
  cursor: not-allowed;
  opacity: 0.65;
}

.progressOuter {
  width: 100%;
  height: 22px;
  background: #e2e8f0;
  border-radius: 20px;
  overflow: hidden;
  margin-top: 8px;
}

.progressInner {
  height: 100%;
  width: 0%;
  background: var(--green);
  text-align: center;
  line-height: 22px;
  font-size: 13px;
  font-weight: bold;
  color: white;
}

.footer {
  text-align: center;
  color: var(--muted);
  margin-top: 18px;
}

@media (max-width: 800px) {
  .grid {
    grid-template-columns: 1fr;
  }

  .header h1 {
    font-size: 22px;
  }

  .card {
    padding: 16px;
  }

  button {
    font-size: 16px;
    padding: 11px 14px;
  }

  .relay-wrap {
    grid-template-columns: 1fr;
  }
}

@media (max-width: 420px) {

```

```

.container {
  width: 96%;
}

.control-grid {
  gap: 6px;
}

.move {
  min-height: 54px;
  font-size: 14px;
}
}
</style>
</head>

<body>

<div class="header">
  <h1>Smart Agriculture Robot</h1>
  <p>Manual and automatic control dashboard</p>
</div>

<div class="container">

<div id="connectionStatus" class="disconnected">
  Disconnected from robot
</div>

<div class="grid">

<div class="card">
  <h2>Mode Control</h2>
  <div class="row">
    <span>Current Mode</span>
    <span class="status" id="modeStatus">MANUAL MODE</span>
  </div>

  <button id="modeButton" class="modeBtn manualMode" onclick="toggleMode()">
    Switch to Auto Mode
  </button>

  <p id="modeHint">Manual mode: use dashboard buttons.</p>
</div>

<div class="card">
  <h2>Motor Status</h2>
  <div class="row">
    <span>Motor</span>
    <span class="status" id="motorStatus">Stopped</span>
  </div>
</div>

<div class="card">
  <h2>Manual Motor Control</h2>
  <p id="manualControlText">Press and hold a button for movement.</p>

  <div class="control-grid">
    <button class="move empty"></button>

    <button class="move manualBtn"
      onmousedown="startMotor('forward')"
      onmouseup="stopMotor()"
      onmouseleave="stopMotor()"
      ontouchstart="startMotor('forward')"
      ontouchend="stopMotor()">Forward</button>

    <button class="move empty"></button>

    <button class="move manualBtn"
      onmousedown="startMotor('left')"
      onmouseup="stopMotor()"
      onmouseleave="stopMotor()"
      ontouchstart="startMotor('left')"
      ontouchend="stopMotor()">Left</button>

    <button class="move stopBtn manualBtn" onclick="stopMotor()">Stop</button>

    <button class="move manualBtn"
      onmousedown="startMotor('right')"
      onmouseup="stopMotor()"
      onmouseleave="stopMotor()"
      ontouchstart="startMotor('right')"
      ontouchend="stopMotor()">Right</button>

    <button class="move empty"></button>

    <button class="move manualBtn"
      onmousedown="startMotor('backward')"
      onmouseup="stopMotor()"
      onmouseleave="stopMotor()"
      ontouchstart="startMotor('backward')"
      ontouchend="stopMotor()">Backward</button>

    <button class="move empty"></button>
  </div>
</div>

<div class="card">
  <h2>Relay Control</h2>

  <div class="relay-wrap">
    <div>
      <div class="row">
        <span>Crops Cutting</span>
        <span class="status" id="relay1Status">OFF</span>
      </div>
      <button id="relay1Button" class="relayBtn relayOff" onclick="toggleRelay1()">
        Crops Cutting OFF
      </button>
    </div>

    <div>
      <div class="row">
        <span>Spray Water</span>
        <span class="status" id="relay2Status">OFF</span>
      </div>
      <button id="relay2Button" class="relayBtn relayOff" onclick="toggleRelay2()">
        Spray Water OFF
      </button>
    </div>
  </div>

</div>

<div class="card">
  <h2>Temperature and Humidity</h2>
  <div class="row">

```

```

        <span>Temperature</span>
        <span class="status" id="temperatureC">0 C</span>
    </div>
    <div class="row">
        <span>Humidity</span>
        <span class="status" id="humidity">0%</span>
    </div>
    <div class="row">
        <span>DHT22 Status</span>
        <span class="status" id="dhtStatus">Unknown</span>
    </div>
</div>

<div class="card">
    <h2>IR Sensors</h2>
    <div class="row">
        <span>Left IR</span>
        <span class="status" id="leftIRStatus">NOT DETECTED</span>
    </div>
    <div class="row">
        <span>Right IR</span>
        <span class="status" id="rightIRStatus">NOT DETECTED</span>
    </div>
</div>

<div class="card">
    <h2>Ultrasonic Sensor</h2>
    <div class="row">
        <span>Distance</span>
        <span class="status" id="distanceCm">0 cm</span>
    </div>
    <div class="row">
        <span>Status</span>
        <span class="status" id="ultrasonicStatus">Unknown</span>
    </div>
    <div class="row">
        <span>Auto Stop Distance</span>
        <span class="status" id="autoStopDistance">15 cm</span>
    </div>
</div>

<div class="card">
    <h2>Soil Moisture Sensor</h2>
    <div class="row">
        <span>Moisture</span>
        <span class="status" id="soilPercent">0%</span>
    </div>
    <div class="row">
        <span>Status</span>
        <span class="status" id="soilStatus">Unknown</span>
    </div>
    <div class="row">
        <span>Raw Value</span>
        <span class="status" id="soilRaw">0</span>
    </div>

    <div class="progressOuter">
        <div class="progressInner" id="soilProgress">0%</div>
    </div>
</div>

<div class="card">
    <h2>Rain Sensor</h2>
    <div class="row">
        <span>Rain Level</span>
        <span class="status" id="rainPercent">0%</span>
    </div>
    <div class="row">
        <span>Status</span>
        <span class="status" id="rainStatus">Unknown</span>
    </div>
    <div class="row">
        <span>Raw Value</span>
        <span class="status" id="rainRaw">0</span>
    </div>

    <div class="progressOuter">
        <div class="progressInner" id="rainProgress">0%</div>
    </div>
</div>
</div>

<div class="footer">
    <p>WiFi AP: <b>Smart Agriculture Robot</b></p>
    <p>Password: <b>12345678</b></p>
    <p>Open browser: <b>192.168.4.1</b></p>
</div>
</div>

<script>
var socket;
var reconnectInterval = 2000;

var relay1State = false;
var relay2State = false;
var autoMode = false;

function connectWebSocket() {
    socket = new WebSocket('ws://' + window.location.hostname + ':81/');

    socket.onopen = function() {
        document.getElementById('connectionStatus').className = 'connected';
        document.getElementById('connectionStatus').textContent = 'Connected to robot';
    };

    socket.onclose = function() {
        document.getElementById('connectionStatus').className = 'disconnected';
        document.getElementById('connectionStatus').textContent = 'Disconnected from robot. Reconnecting...';
        setTimeout(connectWebSocket, reconnectInterval);
    };

    socket.onerror = function() {
        document.getElementById('connectionStatus').className = 'disconnected';
        document.getElementById('connectionStatus').textContent = 'Connection error';
    };

    socket.onmessage = function(event) {
        var data = JSON.parse(event.data);

        if (typeof data.motor !== 'undefined') {
            document.getElementById('motorStatus').textContent = data.motor;
        }

        if (typeof data.autoMode !== 'undefined') {
            autoMode = data.autoMode;
        }
    };
}

```

```

    updateModeUI();
}

if (typeof data.relay1 !== 'undefined') {
    relay1State = data.relay1;
    updateRelay1UI();
}

if (typeof data.relay2 !== 'undefined') {
    relay2State = data.relay2;
    updateRelay2UI();
}

if (typeof data.leftIR !== 'undefined') {
    document.getElementById('leftIRStatus').textContent =
        data.leftIR ? 'DETECTED' : 'NOT DETECTED';
}

if (typeof data.rightIR !== 'undefined') {
    document.getElementById('rightIRStatus').textContent =
        data.rightIR ? 'DETECTED' : 'NOT DETECTED';
}

if (typeof data.soilPercent !== 'undefined') {
    updateSoilUI(data.soilPercent, data.soilRaw, data.soilStatus);
}

if (typeof data.rainPercent !== 'undefined') {
    updateRainUI(data.rainPercent, data.rainRaw, data.rainStatus);
}

if (typeof data.distanceCm !== 'undefined') {
    updateUltrasonicUI(data.distanceCm, data.ultrasonicStatus);
}

if (typeof data.temperatureC !== 'undefined') {
    document.getElementById('temperatureC').textContent =
        Number(data.temperatureC).toFixed(1) + ' C';
}

if (typeof data.humidity !== 'undefined') {
    document.getElementById('humidity').textContent =
        Number(data.humidity).toFixed(0) + '%';
}

if (typeof data.dhtStatus !== 'undefined') {
    document.getElementById('dhtStatus').textContent = data.dhtStatus;
}
};

function sendCommand(command) {
    if (socket && socket.readyState === WebSocket.OPEN) {
        socket.send(JSON.stringify(command));
    }
}

function startMotor(direction) {
    if (autoMode) return;
    sendCommand({motor: direction});
}

function stopMotor() {
    if (autoMode) return;
    sendCommand({motor: 'stop'});
}

function toggleMode() {
    autoMode = !autoMode;
    sendCommand({autoMode: autoMode});
    updateModeUI();
}

function toggleRelay1() {
    relay1State = !relay1State;
    sendCommand({relay1: relay1State});
    updateRelay1UI();
}

function toggleRelay2() {
    relay2State = !relay2State;
    sendCommand({relay2: relay2State});
    updateRelay2UI();
}

function updateModeUI() {
    var modeStatus = document.getElementById('modeStatus');
    var modeButton = document.getElementById('modeButton');
    var modeHint = document.getElementById('modeHint');
    var manualControlText = document.getElementById('manualControlText');
    var manualButtons = document.getElementsByClassName('manualBtn');

    if (autoMode) {
        modeStatus.textContent = 'AUTO MODE';
        modeButton.textContent = 'Switch to Manual Mode';
        modeButton.className = 'modeBtn autoMode';
        modeHint.textContent = 'Auto mode: robot follows IR sensors and stops for obstacles.';
        manualControlText.textContent = 'Manual buttons are disabled in Auto Mode.';

        for (var i = 0; i < manualButtons.length; i++) {
            manualButtons[i].classList.add('disabledBtn');
        }
    } else {
        modeStatus.textContent = 'MANUAL MODE';
        modeButton.textContent = 'Switch to Auto Mode';
        modeButton.className = 'modeBtn manualMode';
        modeHint.textContent = 'Manual mode: use dashboard buttons.';
        manualControlText.textContent = 'Press and hold a button for movement.';

        for (var i = 0; i < manualButtons.length; i++) {
            manualButtons[i].classList.remove('disabledBtn');
        }
    }
}

function updateRelay1UI() {
    var status = document.getElementById('relay1Status');
    var button = document.getElementById('relay1Button');

    if (relay1State) {
        status.textContent = 'ON';
        button.textContent = 'Crops Cutting ON';
        button.className = 'relayBtn relayOn';
    } else {
        status.textContent = 'OFF';
        button.textContent = 'Crops Cutting OFF';
        button.className = 'relayBtn relayOff';
    }
}

```

```

    }
}

function updateRelay2UI() {
    var status = document.getElementById('relay2Status');
    var button = document.getElementById('relay2Button');

    if (relay2State) {
        status.textContent = 'ON';
        button.textContent = 'Spray Water ON';
        button.className = 'relayBtn relayOn';
    } else {
        status.textContent = 'OFF';
        button.textContent = 'Spray Water OFF';
        button.className = 'relayBtn relayOff';
    }
}

function updateSoilUI(percent, raw, statusText) {
    document.getElementById('soilPercent').textContent = percent + '%';
    document.getElementById('soilRaw').textContent = raw;
    document.getElementById('soilStatus').textContent = statusText;

    var progress = document.getElementById('soilProgress');
    progress.style.width = percent + '%';
    progress.textContent = percent + '%';
}

function updateRainUI(percent, raw, statusText) {
    document.getElementById('rainPercent').textContent = percent + '%';
    document.getElementById('rainRaw').textContent = raw;
    document.getElementById('rainStatus').textContent = statusText;

    var progress = document.getElementById('rainProgress');
    progress.style.width = percent + '%';
    progress.textContent = percent + '%';
}

function updateUltrasonicUI(distance, statusText) {
    var distanceElement = document.getElementById('distanceCm');

    if (distance < 0) {
        distanceElement.textContent = 'Out of Range';
    } else {
        distanceElement.textContent = Number(distance).toFixed(1) + ' cm';
    }

    document.getElementById('ultrasonicStatus').textContent = statusText;
}

document.addEventListener('keydown', function(event) {
    if (autoMode) return;

    switch(event.key) {
        case 'ArrowUp':
        case 'W':
        case 'w':
            startMotor('forward');
            break;

        case 'ArrowDown':
        case 'S':
        case 's':
            startMotor('backward');
            break;

        case 'ArrowLeft':
        case 'A':
        case 'a':
            startMotor('left');
            break;

        case 'ArrowRight':
        case 'D':
        case 'd':
            startMotor('right');
            break;
    }
});

document.addEventListener('keyup', function(event) {
    if (autoMode) return;

    switch(event.key) {
        case 'ArrowUp':
        case 'ArrowDown':
        case 'ArrowLeft':
        case 'ArrowRight':
        case 'W':
        case 'w':
        case 'S':
        case 's':
        case 'A':
        case 'a':
        case 'D':
        case 'd':
            stopMotor();
            break;
    }
});

document.addEventListener('contextmenu', function(event) {
    if (event.target.classList.contains('move')) {
        event.preventDefault();
        return false;
    }
});

window.onload = function() {
    connectWebSocket();
    updateModeUI();
    updateRelay1UI();
    updateRelay2UI();
};
</script>

</body>
</html>
)rawliteral";

    return page;
}

// ----- Web Server Handler -----
void handleRoot() {
    server.send(200, "text/html", HTMLPage());
}

```

```

// ----- Setup -----
void setup() {
  Serial.begin(115200);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  pinMode(RELAY1_PIN, OUTPUT);
  pinMode(RELAY2_PIN, OUTPUT);

  pinMode(LEFT_IR_PIN, INPUT);
  pinMode(RIGHT_IR_PIN, INPUT);

  pinMode(SOIL_SENSOR_PIN, INPUT);
  pinMode(RAIN_SENSOR_PIN, INPUT);

  pinMode(ULTRASONIC_TRIG_PIN, OUTPUT);
  pinMode(ULTRASONIC_ECHO_PIN, INPUT);

  digitalWrite(ULTRASONIC_TRIG_PIN, LOW);

  analogReadResolution(12);
  analogSetPinAttenuation(SOIL_SENSOR_PIN, ADC_11db);
  analogSetPinAttenuation(RAIN_SENSOR_PIN, ADC_11db);

  stopCar();
  setRelay1(false);
  setRelay2(false);

  // Start LCD on custom I2C pins.
  Wire.begin(I2C_SDA, I2C_SCL);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Smart Agri Bot");
  lcd.setCursor(0, 1);
  lcd.print("Starting...");

  // Start DHT22.
  dht.begin();

  readAllSensors();

  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);

  server.on("/", handleRoot);
  server.begin();

  WebSocket.begin();
  WebSocket.onEvent(WebSocketEvent);

  Serial.println("HTTP server started");
  Serial.println("WebSocket server started on port 81");
  Serial.print("WiFi Name: ");
  Serial.println(ssid);
  Serial.print("IP address: ");
  Serial.println(WiFi.softAPIP());

  delay(1000);
  lcd.clear();
}

// ----- Loop -----
void loop() {
  server.handleClient();
  WebSocket.loop();

  if (millis() - lastSensorReadTime >= 300) {
    lastSensorReadTime = millis();
    readAllSensors();
  }

  autoModeControl();
  updateLCDDisplay();

  if (millis() - lastDashboardSendTime >= 1000) {
    lastDashboardSendTime = millis();
    sendStatus(255);
  }

  delay(5);
}

```